Restaurant Web Application

team of 3 - specification draft

Team members: Horhat Ruxandra, Spineanu Casinana, Micea Christian
Git link: https://github.com/rux28/restaurant

Description: This application digitizes the daily operations of a restaurant for both customers and staff. Clients can browse the menu, order food for delivery and book a table online. Waiters can manage their tables, payments, tips and work hours. Managers get full control over staff, restaurant policies and client access.

Technologies used:

- **Frontend:** React + Vite, Bootstrap (react-bootstrap), React Router, Axios, STOMP/SockJS for real-time updates
- **Backend:** Java 21, Spring Boot 4 (Web, Data JPA, Security, WebSocket, Mail, Validation), Lombok
- **Database:** PostgreSQL on Neon (H2 for offline testing)
- **Authentication:** Firebase Auth, verified in the backend with firebase-admin
- **Hosting:** Firebase Hosting for the frontend, Render for the backend
- **Storage:** Cloudinary for images and videos
- **Email:** Java Mail Sender with a Gmail app password or Mailtrap
- **Tools:** IntelliJ IDEA, WebStorm, Git/GitHub, Postman

Actors:

Actor	Type	Description
Client	Primary	A customer who uses the public side of the app to view the menu, reserve tables and order food
Waiter	Primary	A staff member who serves tables, takes orders, closes bills and records tips
Manager	Primary	A staff member with full rights over staff accounts, menu, restaurant policies
Kitchen	Secondary	Receives the orders sent by waiters and prepares them

Payment provider	External System	Processes online payments
Email service	External System	Sends confirmations/notifications

Use Cases:

UC-1: Browse the menu

- **Actor:** Client
- **User story:** As a client, I can browse categorized menu items with prices, descriptions and dietary tags so that I can choose what to order.
- **Preconditions:** The menu contains at least one visible item.
- **Main flow:**
    1. The client opens the Menu page.
    2. The system shows items grouped by category, each with name, price, description and dietary tags.
    3. The client selects a category or a dietary tag filter.
    4. The system shows only the matching items.
- **Alternative flow:** No items match the filter → the system shows the message "No items found".
- **Postconditions:** The client has seen the items and can add them to a takeaway order.

UC-2: Reserve a table

- **Actor:** Client (secondary: Email service)
- **User story:** As a client, I can select a date, time and party size to reserve a table in advance.
- **Preconditions:** The client is logged in.
- **Main flow:**
    - The client opens the Reservations page.
    - The client selects date, time and party size.
    - The system checks table availability for that slot.
    - The client confirms the reservation.
    - The system saves the reservation and sends a confirmation email.
- **Alternative flows:**
    - No table is free for that slot → the system shows the slot as unavailable and suggests other times.
- **Postconditions:** The reservation is saved and the table is marked reserved for that slot.

UC-3: Place a takeaway order

- **Actor:** Client (secondary: Payment provider, Kitchen)
- **User story:** As a client, I can select items for pickup, set a pickup time and pay online.
- **Preconditions:** The client is logged in and is not banned.
- **Main flow:**
    - The client adds menu items to the cart.
    - The client chooses a pickup time.
    - The client pays online through the payment provider.
    - The system saves the order and sends it to the kitchen.
    - The client receives an order confirmation.
- **Alternative flows:**
    - The payment fails → the order is not saved and the client can retry.
    - The chosen pickup time is outside opening hours → the system asks for another time.
- **Postconditions:** The order is paid and visible to the kitchen.

UC-4: Claim tables on the floor plan

- **Actor:** Waiter
- **User story:** As a waiter, I can view the table floor plan and claim tables for my shift.
- **Preconditions:** The waiter is logged in and clocked in.
- **Main flow:**
    1. The waiter opens the floor plan.
    2. The system shows all tables and their status (free, claimed by the waiter, occupied, reserved).
    3. The waiter selects one or more free tables and claims them.
    4. The system assigns the tables to the waiter.
- **Alternative flow:** The table is already claimed by another waiter → the system refuses the claim and shows who owns it.
- **Postconditions:** The tables appear as assigned to the waiter for the current shift.

UC-5: Enter a dine-in order

- **Actor:** Waiter (secondary: Kitchen)
- **User story:** As a waiter, I can add or modify items on an active table order and send them to the kitchen.
- **Preconditions:** The waiter has claimed the table.
- **Main flow:**
    1. The waiter selects one of their tables.
    2. The system opens the table's active order (or creates a new one).
    3. The waiter adds, removes or changes the quantity of items.
    4. The waiter sends the order to the kitchen.
    5. The system marks the items as sent.
- **Alternative flow:** An item is no longer available → the system blocks it and notifies the waiter.
- **Postconditions:** The kitchen receives the new items for that table.

UC-6: Settle the bill and record the tip

- **Actor:** Waiter
- **User story:** As a waiter, I can close out a table check, select the payment method and log the tip received.
- **Preconditions:** The table has an active order with at least one item.
- **Main flow:**
    1. The waiter opens the table's bill.
    2. The system shows the total.
    3. The waiter selects the payment method (cash or card).
    4. The waiter enters the tip amount (0 if none).
    5. The waiter closes the bill.
    6. The system records the payment and tip and marks the table as free.
- **Alternative flow:** The waiter enters an invalid tip (e.g., negative) → the system shows an error.
- **Postconditions:** The payment and tip are stored and included in tip and sales reports.

UC-7: Manage staff accounts

- **Actor:** Manager
- **User story:** As a manager, I can add, disable, or assign roles to staff members (Waiter, Kitchen, Manager).
- **Preconditions:** The manager is logged in.
- **Main flow:**
    1. The manager opens the Staff page.
    2. The system lists all staff accounts with their roles and status.
    3. The manager adds a new account, changes a role or disables an account.
    4. The system saves the change.
- **Alternative flow:** The email is already in use → the system refuses to create the account.
- **Postconditions:** The staff list is updated; disabled accounts can no longer log in.

UC-8: Configure the tip policy

- **Actor:** Manager
- **User story:** As a manager, I can switch between Pooled Tips (equal split) and Individual Tips (per waiter).
- **Preconditions:** The manager is logged in.
- **Main flow:**
    1. The manager opens Restaurant Settings.
    2. The system shows the current tip policy.
    3. The manager selects Pooled or Individual tips.
    4. The system saves the policy and applies it to tips recorded from now on.
- **Postconditions:** Tip reports are calculated using the new policy.

UC-9: Ban a client

- **Actor:** Manager
- **User story:** As a manager, I can flag or block client accounts or phone numbers from making future reservations or orders.
- **Preconditions:** The manager is logged in; the client has an account or a known phone number.
- **Main flow:**
    1. The manager searches for the client.
    2. The manager selects Ban and optionally adds a reason.
    3. The system marks the client as banned.
    4. The banned client appears in a list visible to staff.
- **Alternative flow:** The manager unbans a client → the client can reserve and order again.
- **Postconditions:** The banned client can no longer reserve tables or place orders online.

